Subject : Programming for Problem Solving Lab.

Unit 2 : Introduction to C Programming

Program : B.Tech. - CSE

Course Code : **CSCR 1501**

Semester : 1st

Faculty Teaching : Dr. Shree Harsh Attri

Designation : Associate Professor

Department : Computer Sc. & Engineering

Sharda University, Greater Noida, U.P

श्री हर्ष अत्री

Contents

C. Introduction to C Programming Language, Data Types, Variables, Constants, Identifiers and Keywords, Operators and Expressions

What is Program?

- Instructions for the hardware.
  - Actions to be performed
- A set of instructions is called a program.
  - Driving force behind the computer
  - Without a program, there is no computer
    - And computer becomes collection of Useless Hardware
- 2 purposes:
  - Tell the computer what to do
  - Tell other people what we want the computer to do.

Introduction to C Programming

WHAT IS C PROGRAMMING LANGUAGE ?

- C language is a structure/procedure oriented, middle/high level programming language developed at Bell Laboratories in 1972 by Dennis Ritchie.
- C language was invented for implementing UNIX operating system.
- In 1978, Dennis Ritchie and Brian Kernighan published the first edition “The C Programming Language”.
- Also, C language is an ANSI/ISO standard and powerful programming language for developing real time applications
- C language has been developed using **assembly level language.**

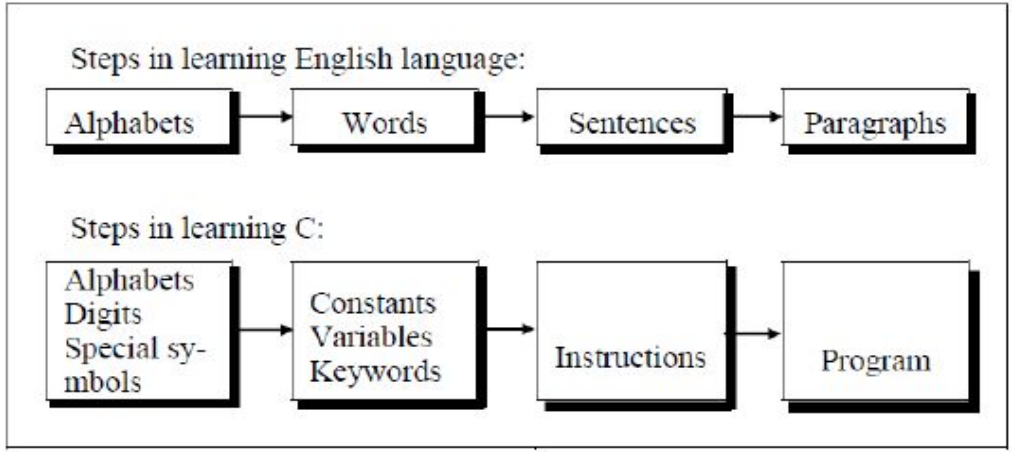
Introduction to C Programming

Learning the Language

There is a close similarity between learning Hindi/English language and learning C language.

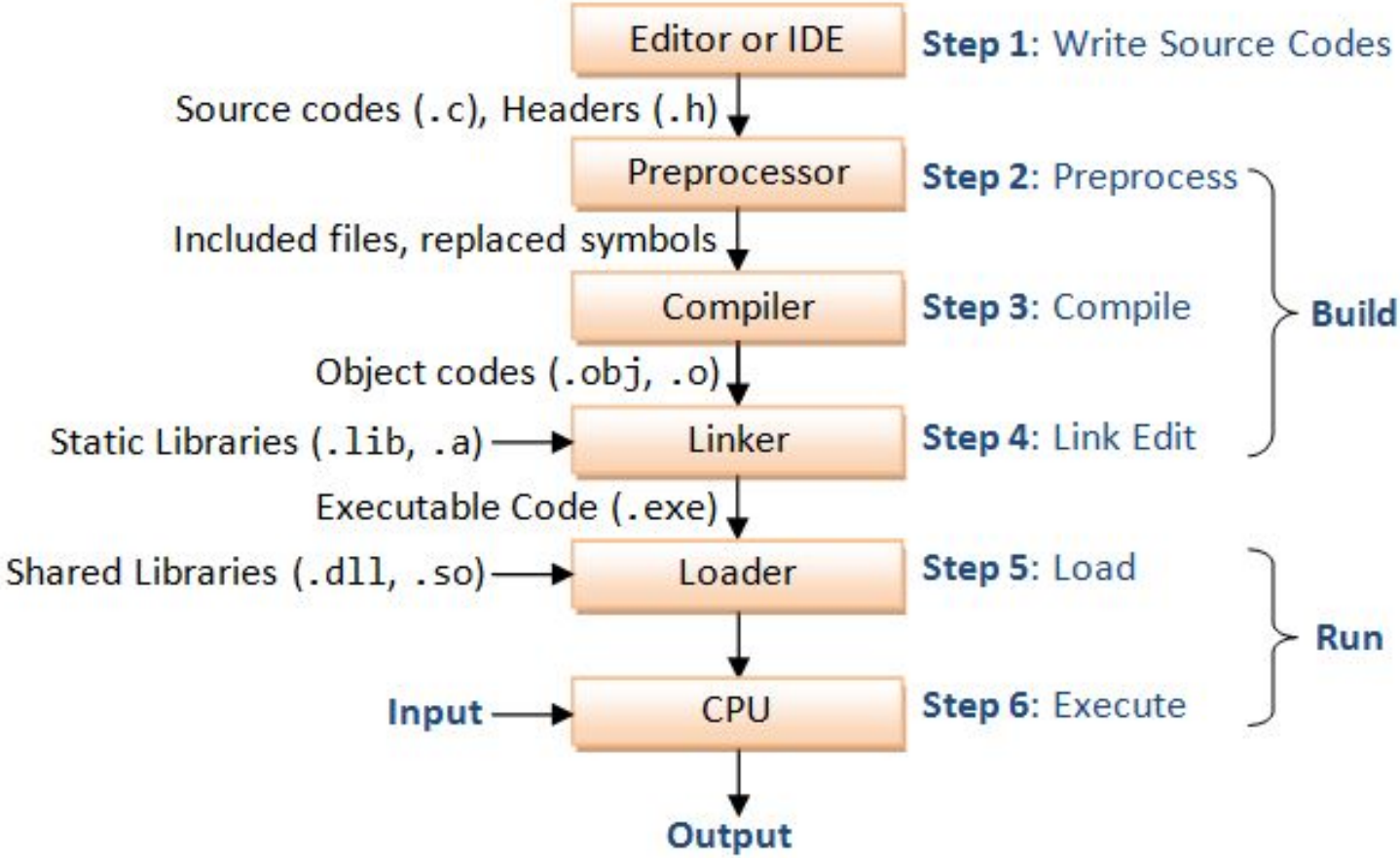
The classical method of learning Hindi/English is to *first learn the alphabets used in the language, then learn to combine these alphabets to form words, which in turn are combined to form sentences and sentences are combined to form paragraphs*.

Learning C is similar and easier. We must first know what ***alphabets, numbers and special symbols*** are used in C, then how using them ***constants, variables and keywords*** are constructed to form an **instruction**. The group of instructions form a **program.**



Introduction to C Programming

Execution of C Program



Introduction to C Programming

Where C Programming Language is used ?

- C language is used to develop system applications that forms major portion of operating systems such as Windows, UNIX and Linux.

- Moreover, if one is to extend the operating system to work with new devices one needs to write device driver programs. These programs are exclusively written in C.

- C is popular because it is **reliable**, **simple** and **easy** to use. Even today when it comes to performance (speed of execution) nothing beats C.

Below are more examples of uses of C language.

- Database systems
- Graphics packages
- Word processors
- Spread sheets
- Compilers and Assemblers
- Network drivers
- Interpreters

C Basics

C Character Sets

- As every language contains a set of characters used to construct words, statements, etc., C language also has a set of characters which include **alphabets, digits**, and **special symbols**.
- C language supports a total of 256 characters.
- Every C program contains statements. These statements are constructed using words and these words are constructed using characters from C character set.
- C language character set contains the following set of characters.

1. Alphabets
2. Digits
3. Special Symbols

C Basics

C Character Sets

1. **Alphabets:** C language supports all the alphabets from the English language. Lower and upper case letters together support 52 alphabets.
   lower case letters - **a to z**
   UPPER CASE LETTERS - **A to Z**

2. **Digits:** C language supports 10 digits which are used to construct numerical values in C language.
   Digits - **0, 1, 2, 3, 4, 5, 6, 7, 8, 9**

3. **Special Symbols:** C language supports a rich set of special symbols that include symbols to perform *mathematical operations, to check conditions, white spaces, backspaces, and other special symbols*
   Special Symbols - **~ @ # $ % ^ & * ( ) _ - + = { } [ ] ; : ' " / ? . > , < \ | tab newline space NULL etc.,**

Every character in C language has its equivalent ASCII (American Standard Code for Information Interchange) value.

C Basics

Tokens in C

A collection of all the ***keywords, identifiers, operators, special symbols, constants, strings, and data values*** are called tokens.
Tokens are used to construct C programs and they are said to the basic building blocks of a C program.

C tokens are of six types.

1. Keywords (eg: int, while),
2. Identifiers (eg: student_name, roll_number),
3. Constants (eg: 10, 20),
4. Strings (eg: “total”, “hello”),
5. Special symbols (eg: (), {}),
6. Operators (eg: +, /,-,*)

C Basics

Identifiers and Keywords

- The **identifier** is a user-defined name of an entity to identify it uniquely during the program execution.
- **Keywords** are the reserved words with predefined meaning which already known to the compiler.

Keywords	Identifier
• Predefined-word • Must be written in lowercase only • Has fixed meaning and meaning has already been explained to the C compiler • Combination of alphabetic characters • Keywords can not be used as user-defined names like variable, functions, arrays, pointers, etc. • There are 32 keywords in C Programming int, float, char, double, long, short, if, else, switch, case, default, for, while, do, return, void, auto, extern, const, extern, register, signed, unsigned, break, continue, goto, enum, sizeof, struct, typedef, union	• User-defined word • Can written in lowercase and uppercase • Must be meaningful in the program and whose meaning not explained to the C compiler. • Combination of alphanumeric characters • Underscore character is considered as a letter • Eg.: int marks, char student_Name[30];

C Basics

Rules for Creating Identifiers

1. An identifier can contain **letters** (UPPERCASE and lowercase), **numerics** & **underscore** symbol only.
2. An identifier should not start with a numerical value. It can start with a letter or an underscore.
3. Any special symbols should not be used in between the identifier even whitespace. However, the only underscore symbol is allowed.
4. Keywords should not be used as identifiers.
5. There is no limit for the length of an identifier. However, the compiler considers the first 31 characters only.
6. An identifier must be unique in its scope.

C Basics

Data types:

- Data types in C language are defined as the data storage format that a variable can store a data to perform a specific operation.
- Data types are used to define a variable before to use in a program.
- Size of variable, constant and array are determined by data types.

Basic Data Types

Type	Data types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	enum
Void Data Type	void

char – char defines a character used to store a single character.

Example: char ch = ‘a’;

int – int is used to store integer numbers.

Example: int count = 5;

float – float is used to define floating-point numbers with single precision.

Example: float x = 10.12; (32 bits are used to store)

double – double is used to define floating-point numbers with double precision.

Example: double num = 10.9999; (64 bits are used to store)

C Basics

Range of Data types

Different data types also have different value range. The storage size of basic data types changes according to16-bit or 32-bit compiler for 16-bit or 32-bit processors respectively.

- The storage size of **int** data type is 2 or 4 bytes depend upon the compiler.
- **char** data type allows a variable to store only one character. Storage size of character data type is 1 byte.
- **float** data type allows a variable to store decimal values. Storage size of float data type is 4 bytes.
- **double** data type is also same as float data type which allows up-to 10 digits after decimal.

C Basics

Size of Primary Data Types

Type	Range	Size (in bytes)
unsigned char	0 to 255	1
signed char or char	-128 to +127	1
unsigned int	0 to 65535	2
signed int or int	-32,768 to +32767	2
unsigned short int	0 to 65535	2
signed short int or short int	-32,768 to +32767	2
unsigned long int	0 to +4,294,967,295	4
signed long int or long int	-2,147,483,648 to +2,147,483,647	4
long double	3.4E-4932 to 1.1E+4932	10
double	1.7E-308 to 1.7E+308	8
float	3.4E-38 to 3.4E+38	4

C Basics

Constants in C

- A constant is a named memory location which holds only one value throughout the program execution..
- C Constants are also like normal variables. But, only difference is, constant values can't be modified by the program once they are defined. Constants may be belonging to any of the data type.

Types of Constant in C

- Integer constants
- Real or Floating point constants
- Character constants
- String constants
- Octal & Hexadecimal constants
- Backslash character constants

C Basics

Types of Constants in C

1. **Integer constants:**
   - An integer constant must have at least one digit.
   - It must not have a decimal point.
   - It can either be positive or negative.
   - No commas or blanks are allowed within an integer constant.
   - If no sign precedes an integer constant, it is assumed to be positive.
   - The allowable range for integer constants is -32768 to 32767.

2. **Real or Floating point constants**
   - A real constant must have at least one digit
   - It must have a decimal point
   - It could be either positive or negative
   - If no sign precedes an integer constant, it is assumed to be positive.
   - No commas or blanks are allowed within a real constant.
   - The range of real constants expressed in exponential form is between -3.4e38 to 3.4e38.

C Basics

Types of Constants in C

3. **Character constants**

   A character constant is a single alphabet, a single digit or a single special symbol enclosed within single quotes.

   The maximum length of a character constant is 1 character.

   String constants are enclosed within double quotes.

   Example: **'A' , '2', '+'**

4. **String constants**

   A string constant is a collection of characters, digits, special symbols and escape sequences that are enclosed in double quotations.

   Example: **"This is C Programming"**

   "123 + $% - * basics of C"

C Basics

Types of Constants in C

5. Backslash Character Constants in C

- There are some characters which have special meaning in C language.
- They should be preceded by backslash symbol to make use of special function of them.
- These characters are called as **Escape sequence characters**. In these characters it consists of two characters, but it is treated as a single character.

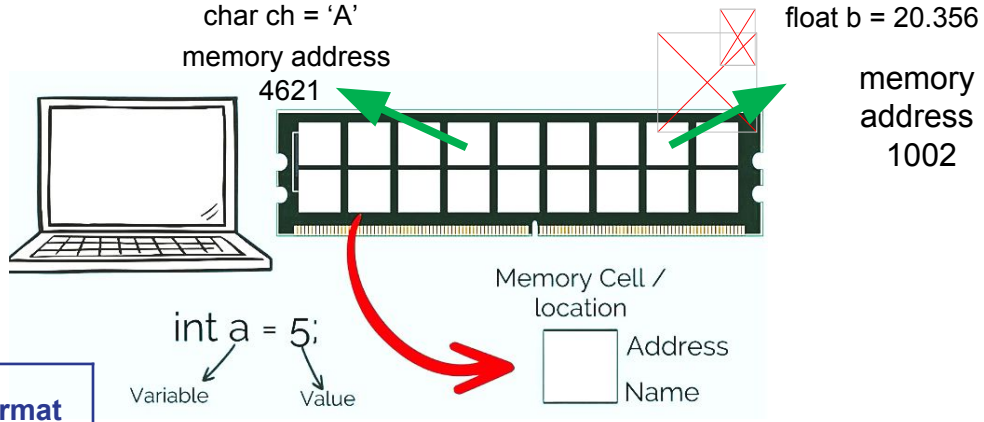
The list of special characters and their purpose

Backslash Character	Meaning
\b	Backspace
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\”	Double quote
\’	Single quote
\\	Backslash
\v	Vertical tab
\a	Alert or bell
\?	Question mark
\N	Octal constant (N is an octal constant)
\XN	Hexadecimal constant (N – hex.dcml cnst)

C Programming Fundamentals

Variable declarations and memory allocation

```
int a = 5;
float b = 20.356;
char ch = 'A';
```



Data type, size, range and format specifiers

Type	Storage size	Value range	Format Specifiers
char	1 byte	-128 to 127 or 0 to 255	%c
int	2 bytes or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647	%d
float	4 bytes	3.4E-38 to 3.4E+38	%f
double	8 bytes	1.7E-308 to 1.7E+308	%lf

Structure of C Programming

C program basically consists of the following parts −

- Preprocessor Commands
- Function
- Variables
- Statements & Expressions
- Comments

```
/* My first C program to print Hello, World! */ ← Comment

#include <stdio.h> ← Pre-processor directive

int main() ← Main function
{
        printf("Hello, World!"); ← Function   } Body of main function

}
```

Structure of C Programming

Formatting input and output

C language supports two formatting functions

printf: used to convert data stored in the program into a text for output.

Syntax:

```
printf("Message");
printf("control string", variable name);
```

For example:

```
printf("value of variable is %d",x);
```

scanf: used to read formatted data from the keyboard

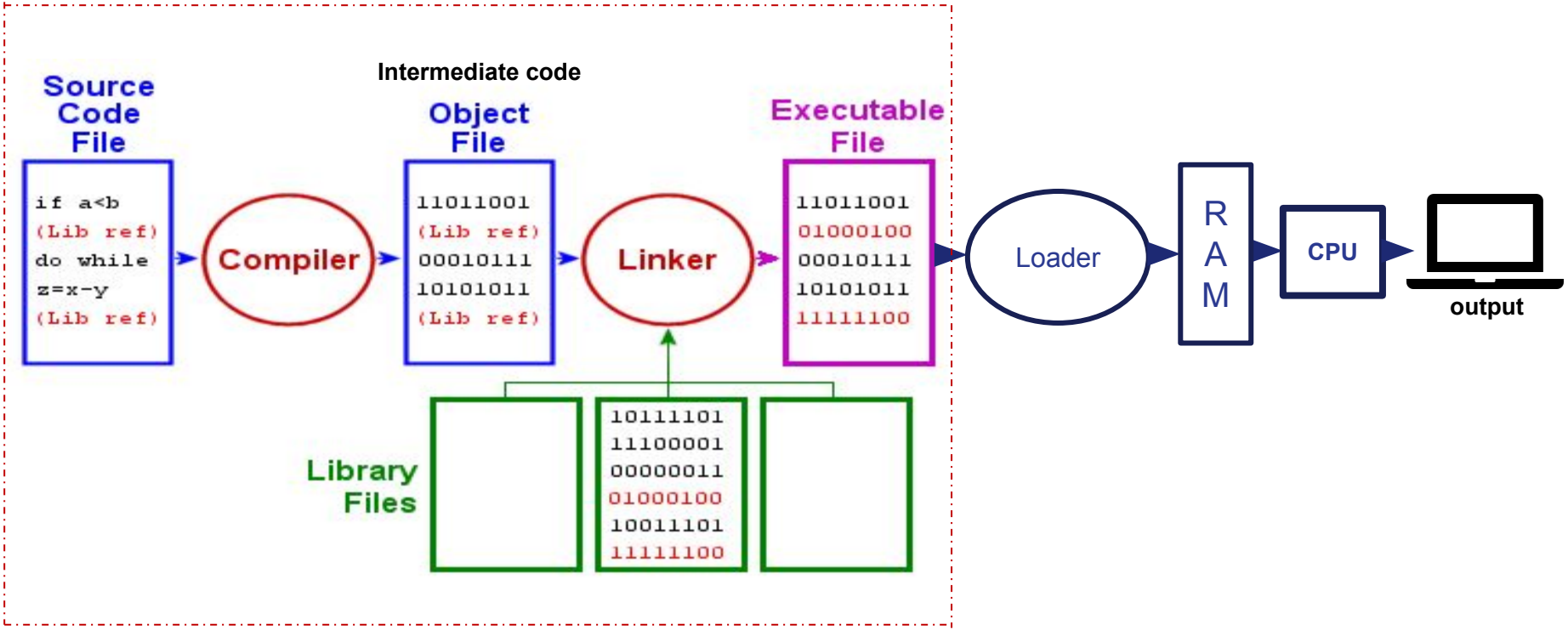
Syntax:

```
scanf("control string", &variable name);
```

For example:

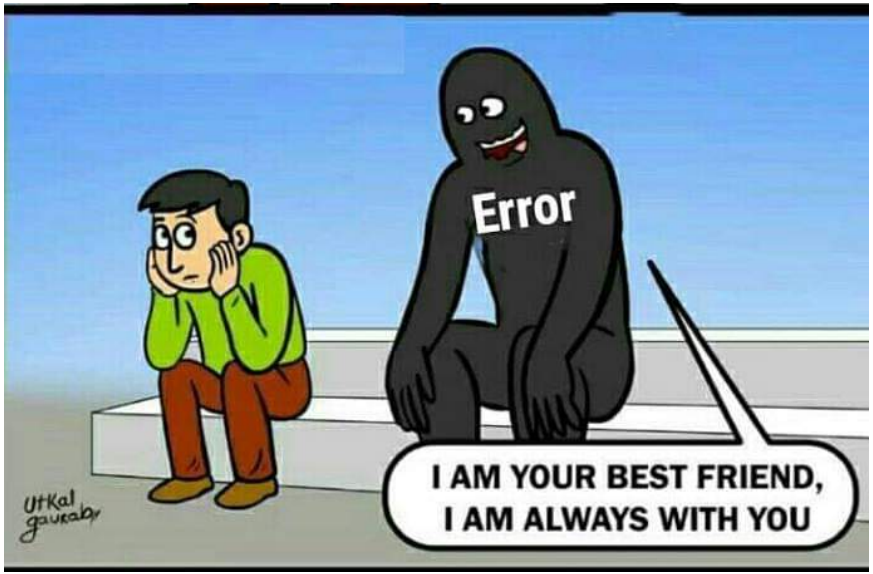
```
scanf("%d", &x);
```

Source Code, Object Code and Executable Code



Integrated Development Environment (IDE)

Programmers are never **ALONE**



Errors in Compilation

Errors are the problems or the faults that occur in the program.

1. **Syntax error**: occurred when programmer does not follow the ***grammar rule*** of the programming language while typing the code.

Example: spelling mistakes, missing of semicolon/parenthesis etc.

```
c = a + b
```

semicolon missing

```
print("C Programming");
```

spelling mistake

2. **Logical error**: produces wrong output but there is no error in program.

In following example, user wants the desired output as **3**

Example:

```
a = 2, b = 4;
c = a + b/2;
```

Logical error

Output : 4

Correct code:

```
a = 2, b = 4;
c = (a + b) / 2;
```

Output : 3

Operators in C

What is Operator in Programming Language?

1. The symbols which are used to perform logical and mathematical operations in any programing language are called Operators.

2. These operators join individual constants and variables to form expressions. Operators, functions, constants and variables are combined together to form expressions.

For example:

```
int x, y, z;
  z = x + y;          // arithmetic operator
```

Types of Operators in C Programming Language

1. Assignment Operators
2. Arithmetic Operators
3. Relational Operators
4. Logical Operators
5. Equality Operators
6. Bit wise Operators
7. Conditional Operators (ternary operators)
8. Unary Operators
9. Comma Operators
10. Sizeof Operators

Assignment Operator

Assignment operators are used to assign the values to the variables.

For example, if the value “10” is to be assigned for the variable “sum”, it can be assigned as “sum = 10;”
There are 2 categories of assignment operators in C language.

1. Simple assignment operator (Example: =)
2. Compound assignment operators (Example: +=, -=, *=, /=, %=, &=, ^=)

Operators	Example/Description	
=	sum = 10;	10 is assigned to variable sum
+=	sum += 10;	This is same as sum = sum + 10
-=	sum -= 10;	This is same as sum = sum – 10
*=	sum *= 10;	This is same as sum = sum * 10
/=	sum /= 10;	This is same as sum = sum / 10
%=	sum %= 10;	This is same as sum = sum % 10
&=	sum&=10;	This is same as sum = sum & 10
^=	sum ^= 10;	This is same as sum = sum ^ 10

Arithmetic Operators

- ✔ C Arithmetic operators are used to perform mathematical calculations like addition, subtraction, multiplication, division and modulus in C programs.
- ✔ Arithmetic operators are +, -, *, /, %

```
#include <stdio.h>
 main()
{
     int a=40,b=20, add,sub,mul,div,mod;
     add = a+b;
     sub = a-b;
     mul = a*b;
     div = a/b;
     mod = a%b;
     printf("Addition of a, b is : %d\n", add);
     printf("Subtraction of a, b is : %d\n", sub);
     printf("Multiplication of a, b is : %d\n", mul);
     printf("Division of a, b is : %d\n", div);
     printf("Modulus of a, b is : %d\n", mod);
}
```

Output:

Addition of a, b is : 60
Subtraction of a, b is : 20
Multiplication of a, b is : 800
Division of a, b is : 2
Modulus of a, b is : 0

Relational and Equality Operators

- ✔ Relational operators are used to find the relation between two variables. i.e. to compare the values of two variables in a C program.
- ✔ Relational operators are >, <, >=, <=,
- ✔ Equality operators are ==, !=

Operators	Example / Description
>	x > y (x is greater than y)
<	x < y (x is less than y)
>=	x >= y (x is greater than or equal to y)
<=	x <= y (x is less than or equal to y)
==	x == y (x is equal to y)
!=	x != y (x is not equal to y)

Relational and Equality Operators

Practical

```
include <stdio.h>
main()
{
int a = 2, b = 2, c = 5;
printf("%d == %d is %d \n", a, b, a == b);
printf("%d == %d is %d \n", a, c, a == c);
printf("%d > %d is %d \n", a, b, a > b);
printf("%d > %d is %d \n", a, c, a > c);
printf("%d < %d is %d \n", a, b, a < b);
printf("%d < %d is %d \n", a, c, a < c);
printf("%d != %d is %d \n", a, b, a != b);
printf("%d != %d is %d \n", a, c, a != c);
printf("%d >= %d is %d \n", a, b, a >= b);
printf("%d >= %d is %d \n", a, c, a >= c);
printf("%d <= %d is %d \n", a, b, a <= b);
printf("%d <= %d is %d \n", a, c, a <= c);
}
```

Logical Operators

Logical operators are used to perform logical operations on the given expressions.
There are 3 logical operators in C language. They are,

- ✔ logical AND (&&)
- ✔ logical OR (||)
- ✔ logical NOT (!)

Operators	Example / Description
&& (logical AND)	(x > 5) && (y < 5) It returns true when both conditions are true
|| (logical OR)	(x >= 10) | |(y >= 10) It returns true when at-least one of the condition is true
! (logical NOT)	!((x > 5) && (y < 5)) It reverses the state of the operand “((x > 5) && (y < 5))” If “((x > 5) && (y < 5))” is true, logical NOT operator makes it false

Logical Operators

Practical

```
#include <stdio.h>
main()
{
int a = 5, b = 5, c = 10, result;
printf("(a == b) && (c > b) is %d \n",  (a == b) && (c > b));
printf("(a == b) && (c < b) is %d \n",  (a == b) && (c < b));
printf("(a == b) || (c < b) is %d \n",  (a == b) || (c < b));
printf("(a != b) || (c < b) is %d \n",  (a != b) || (c < b));
printf("!(a != b) is %d \n",  !(a != b));
printf("!(a == b) is %d \n",  !(a == b));
}
```

Bitwise Operators

- Bitwise operators are used to perform bit operations. Decimal values are converted into binary values which are the sequence of bits and bit wise operators work on these bits.

Truth table for bit wise operation & Bit wise operators:

x	y	x|y	x&y	x^y
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

Below are the bit-wise operators and their name in C language.

& Bitwise AND
| Bitwise OR
~ Bitwise NOT
^ XOR
<< Left Shift
>> Right Shift

Example:

Consider x = 40 and y = 80.
Binary forms of these values are given below:

x = 00101000
y = 01010000

All bit wise operations for x and y are given below.

x & y = 00000000 (binary) = 0 (decimal)
x | y = 01111000 (binary) = 120 (decimal)
~x = 11010111 = -41 (decimal)
x ^ y = 01111000 (binary) = 120 (decimal)
x << 1 = 01010000 (binary) = 80 (decimal)
x >> 1 = 00010100 (binary) = 20 (decimal)

Bitwise Operators

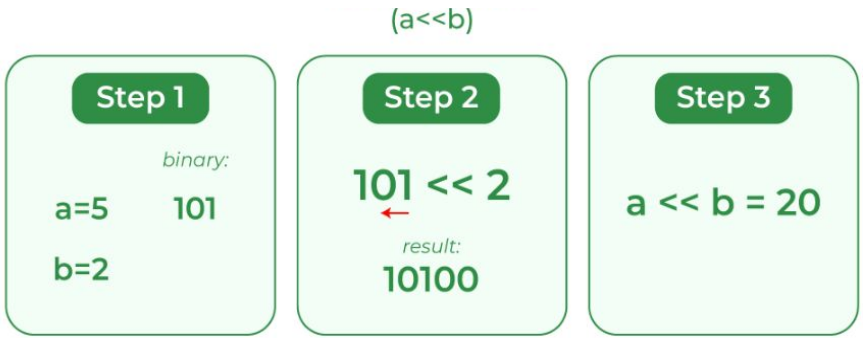
Left Shift (<<) Operator

It is a binary operator that takes two numbers, left shifts the bits of the first operand and the second operand decides the number of places to shift.

Syntax:

a << b; → **a: First Operand**
b: Second Operand

How left shift is done?



left-shifting an integer "**a**" with an integer "**b**" is equivalent to **multiplying a** with **2^b** (2 raised to power b).

Example:
Let's take **a = 5**; which is **101** in Binary Form.

Now, if "*a is left-shifted by 2*" i.e **a = a << 2** then **a** will become **a = a * (2^2)**

Thus, **a = 5 * (2^2) = 20** which can be written as **10100.**

Bitwise Operators

Right Shift (>>) Operator

It is a binary operator that takes two numbers, right shifts the bits of the first operand and the second operand decides the number of places to shift.

Syntax:

a >> b; → **a: First Operand**
b: Second Operand

How right shift is done?

right-shifting an integer "**a**" with an integer "**b**" is equivalent to **dividing a** with **2^b**

Example:
Let's take **a = 5**; which is **101** in Binary Form.

Now, if "*a is right-shifted by 2*" i.e. **a = a >> 2** then **a** will become **a = a / (2^2)**

Thus, **a = 5 / (2^2) = 1** which can be written as **01.**

Bitwise Operators

Practical

```c
#include<stdio.h>
main()
{
    int x = 40, y = 80, a = 5, b = 2;
    printf("Bitwise AND of 40 and 80 is %d\n",x&y);
    printf("Bitwise OR of 40 and 80 is %d\n",x|y);
    printf("Bitwise NOT of 40 is %d\n", ~x);
    printf("Bitwise NOT of 80 is %d\n", ~y);
    printf("Bitwise XOR of 40 and 80 is %d\n", x^y);
    printf("Left shift of a is %d\n",a<<b);
    printf("Right shift of a is %d\n",a>>b);
}
```

Output

```
Bitwise AND of 40 and 80 is 0
Bitwise OR of 40 and 80 is 120
Bitwise NOT of 40 is -41
Bitwise NOT of 80 is -81
Bitwise XOR of 40 and 80 is 120
Left shift of a is 20
Right shift of a is 1
```

Unary Operators

Unary operators act on single operands. C language supports three unary operators:

1. Unary minus (-)
2. Increment Operator
3. Decrement Operator

Unary Minus (-)

When an operand is preceded by a minus sign, the unary operator negates its value.

For Example:

```
int a, b = 10;
a = - (b);
```

Output

a = -10

Increment operators are used to increase the value of the variable by one.

Decrement operators are used to decrease the value of the variable by one in C programs.

Syntax:

Increment operator : ++var_name; (or) var_name++;

Decrement operator : – -var_name; (or) var_name – -;

Example:

Increment operator : ++ i ; i ++ ;

Decrement operator : – – i ; i – – ;

Unary Operators

Difference between pre/post increment & decrement operators in C:

For example:

int x = 10, y;

y = x++;

statement y = x++ is equivalent to writing

y = x;

x = x+1;

whereas y = ++x is equivalent to writing

x = x + 1;

y = x;

above said principle is applied to decrement operators too.

Operator	Operator / Description
Pre increment operator (++i)	value of i is incremented before assigning it to the variable i
Post increment operator (i++)	value of i is incremented after assigning it to the variable i
Pre decrement operator (--i)	value of i is decremented before assigning it to the variable i
Post decrement operator (i--)	value of i is decremented after assigning it to variable i

Unary Operators

Practical

```
#include<stdio.h>
main()
{
     int i,x;
     printf("enter the value of i: ");
     scanf("%d",&i);
      x=i;
     printf("\nvalue of i is %d",i);
     printf("\nvalue of ++i is %d",++i);
     printf("\ndvalue of i++ is %d",i++);
     printf("\n new updated value of i is %d",i);
     printf("\nrevised value of i is %d",x);
     printf("\nvalues of ++i,i++,i--,i is %d %d %d %d",++x,x++,x--,x);
     printf("\n new updated value of i is %d",x);
}
```

Size of Operator

sizeof() operator is used to find the memory space allocated for each C data types.

For example:

```
int x = 10;
unsigned int result;
result = sizeof(x);
```

Output

result = 2; // since x is an integer; it requires 2 bytes of storage space.

Ternary Operator

Ternary operators (? :) are conditional operators and is same as if else control statement in C.

Syntax :

Variable = (Condition? true_value: false_value);

Or

Variable = (expression1 ? expression2 : expression 3)

Example

n1 = 5, n2 = 10,

max = (n1 > n2) ? n1 : n2;

Answer

max = 10

Precedence and Associativity

Precedence : decides how an expression is evaluated and determines the grouping of terms in an expression.
Associativity : used when two operators of same precedence appear in an expression.

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Examples

1. int a = 10, b = 2, x = 0;
   x = a + b * a + 10 / 2 * a;

2. int a = b = c = 7;
   a += b -= c * 10;

Expanded as:

a = a + (b – (c * 10))

References

Web Links:

- https://www.journaldev.com/27602/operators-in-c
- https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-c-c/
- https://fresh2refresh.com/c-programming/c-operators-expressions/c-conditional-operators/